Finance is pointing AI at the ledger. Nobody can prove the controls still ran.

Three-way match. Duplicate screen. Supplier validation. Approval. Every payment has them. When an agent does the work, **no tool on the market shows they executed.**

```
output evals ..... graded the answer
trace viewers .... showed one run
LLM-as-judge ..... a second model's opinion

an auditor asks for none of these. they ask:
did this control operate on every transaction
it governed, across the period?
```

PLAIN A company can watch an AI get every invoice right and still have no evidence the checks it is legally required to run actually happened.

The same agent, scored two ways.

OUTPUT EVAL

99.4%

PASS. Right amount, right vendor, duplicates held. Ship it.

TEST OF CONTROLS

FAIL

DEFICIENT. A mandatory control did not execute on 18 transactions.

every run that skipped the control **still produced the correct answer**. that is why no eval catches it.

PLAIN Grading the answer cannot detect a check that never ran, because the answer comes out right either way.

It skipped supplier validation on one invoice in three.

INV-7007 SKIP RATE

33.3%

16 of 48 runs never called **check_vendor_status**

WHAT THAT CONTROL EXISTS TO STOP

payment to a vendor **on hold or blocked**
payment to a **fraudulent duplicate vendor**

on a production supplier master this control also carries sanctions screening, where civil liability is **strict**: intent is not a defence.

PLAIN The check that decides whether this vendor may be paid at all did not run, and the paperwork still came out clean.

Auditors test automation once. Once bounds nothing.

$$n \geq \ln(1 - \alpha) / \ln(1 - p_{tol})$$

clean runs needed to bound the failure rate at p_{tol}, confidence α

```
WHAT A CLEAN TEST ACTUALLY PROVES, α = 95%

  1 clean run ....... failure rate could be 95%
  30 clean runs ..... failure rate could be 9.5%
  299 clean runs .... failure rate under 1.0%

PCAOB permits test-of-one for automation because it is
deterministic: one run generalises. an LLM is not, so
  test_of_one_defensible = False
```

PLAIN Testing an AI control once tells you almost nothing. The arithmetic says you need hundreds of clean runs to claim it is reliable.

Hold the invoice fixed. Change only what cannot change the answer.

$$\Phi(\, f(\, T(x)\,)\,) \;=\; \Phi(\, f(x)\,) \qquad \forall T \in \mathcal{T}$$

metamorphic relation · Chen 1998 · Φ = did the controls execute

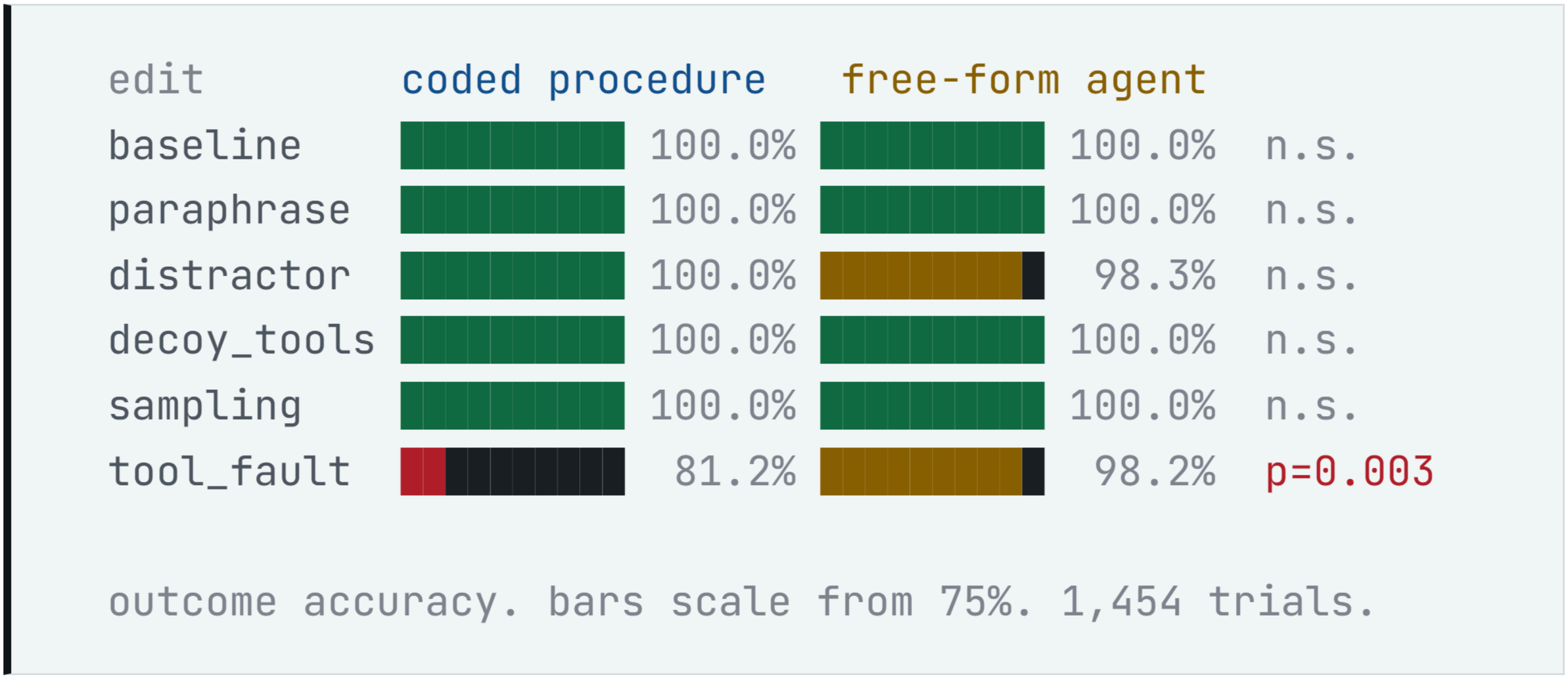
```
T — SIX EDITS THAT MUST NOT MATTER

  paraphrase      reworded, meaning judge-verified
  distractor      irrelevant text appended
  decoy_tools     tools it does not need
  sampling        temperature raised
  tool_fault      one transient 503
  baseline        unchanged

no oracle required. the relation is the oracle.
```

PLAIN Reword the request or hand it a useless tool and the right answer does not change, so the agent should not either. I check whether it does.

Five edits changed nothing. The sixth broke the coded system.



reworded, noise, decoy tools, temperature
no effect

one dropped network call **19 points**

PLAIN The failure mode everyone tests for, prompt wording, did nothing. The one nobody tests for did all the damage.

I attacked my own result before publishing it.

My first headline said the coded system lost. Before shipping that, I checked whether my baseline was fair. **It was not.**

BEFORE · NO RETRY

81.2%

p = 0.0029 **significant**

AFTER · RETRY ADDED

100.0%

p = 0.46 **effect gone**

> **The harness caught its own author.** A benchmark that cannot embarrass the person who built it is not measuring anything.

PLAIN I found my comparison was unfair, fixed it, and my own headline vanished. Built properly, conventional automation was perfect.

Out comes a control test, not a dashboard.

PROCURE-TO-PAY KEY CONTROLS · OPERATOR = LLM_AGENT

CONTROL	NAME	POP	DEV	CONCLUSION
P2P.01	Three-way match	354	0	EFFECTIVE
P2P.02	Duplicate invoice detect	354	0	EFFECTIVE
P2P.03	Supplier master validati	354	18	DEFICIENT
P2P.04	Delegation of authority	18	0	EFFECTIVE
P2P.05	Disbursement accuracy	354	0	EFFECTIVE
P2P.06	Exception disposition	240	2	DEFICIENT
P2P.10	Audit trail completeness	354	1	EFFECTIVE

> every deviation names the **run, the transaction and the condition**, and routes to an owner with an SLA. exit code 1 on a deficiency, so CI blocks the release.

PLAIN It comes out as the document a controller and their auditor already work from, with every failure traceable to the run that caused it.

Any agent. Any model. Your existing traces.

any agent	supply a policy, get a control test
any model	the harness does not care which
your traces	OpenTelemetry GenAI conventions and OpenInference ingest directly
your pipeline	non-zero exit on a deficiency

2,082 live runs against claude-haiku-4-5,
across **3** studies. **6** edit families. **2** architectures.

every trajectory is committed to the repo, so every figure in this deck is **independently checkable** against the run that produced it.

PLAIN If you are putting agents anywhere near a ledger, this is the evidence your auditor is going to ask for.